TTK4135 Helicopterlab

Student 116360
Student 593141

April 12, 2026

Department of Engineering Cybernetics

Contents

1 Optimal Control of Pitch/Travel without Feedback **1**
1.1 The continuous model 1
1.2 The discretized model 1
1.3 The Open Loop Optimization Problem 2
1.3.1 Objective Function 2
1.3.2 Discrete-time dynamics 3
1.3.3 Pitch angle limits: 3
1.4 The weights of the optimization problem 4
1.5 Experimental results 5

2 Optimal Control of Pitch/Travel with Feedback (LQ) **7**
2.1 LQ controller 7
2.2 Model Predictive Control 9
2.3 Experimental results 10
2.4 MATLAB and Simulink 13

3 Optimal Control of Pitch/Travel and Elevation with Feedback **14**
3.1 The continuous model 14
3.2 The discretized model 14
3.3 Optimization Problem 15
3.4 Optimization problem in MATLAB 15
3.5 Experimental results 16
3.6 Decoupled model 18

References **20**

1 Optimal Control of Pitch/Travel without Feedback

1.1 The continuous model

The full model derivation for the helicopter is described both in the labreport for TTK4115 and in [1] so it will not be repeated here. The resulting equations are in equation (1) where e is elevation angle, p is pitch, λ is travel angle and r is the travel-rate.

$$\ddot{e} + K_3 K_{ed}\dot{e} + K_3 K_{ep} e = K_3 K_{ep} e_c \tag{1a}$$
$$\ddot{p} + K_1 K_{pd}\dot{p} + K_1 K_{pp} p = K_1 K_{pp} p_c \tag{1b}$$
$$\dot{\lambda} = r \tag{1c}$$
$$\dot{r} = -K_2 p \tag{1d}$$

Worth noting about these equations is that they not only capture the dynamics of the helicopter itself, but they also capture the dynamics of the PID and PD controllers for e and for p.

For the first lab we disregard elevation and write our state space on the form

$$\dot{x} = Ax + Bu$$

where $x = \begin{bmatrix}\lambda & r & p & \dot{p}\end{bmatrix}^\top$ and $u = p_c$. Using the equations in (1) we end up with the following parameters for A and B

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -K_2 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -K_1 K_{pp} & -K_1 K_{pd} \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 0 \\ 0 \\ K_1 K_{pp} \end{bmatrix}$$

1.2 The discretized model

To obtain a discrete model we used forward Euler discretization on the matrices from the last section. Forward euler discretization is given by $A_d = I + \Delta t A,\ B_d = \Delta t B$. The state space can now be formulated on the form $x_{k+1} = A_d x_k + B_d u_k$ with A_d and B_d given by:

$$A_d = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & -\Delta t K_2 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & -\Delta t K_1 K_{pp} & 1 - \Delta t K_1 K_{pd} \end{bmatrix}$$

$$B_d = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \Delta t K_1 K_{pp} \end{bmatrix}$$

1.3 The Open Loop Optimization Problem

In this lab, we aim to calculate an optimal trajectory for the helicopter to move from travel angle $\lambda_0 = \pi$ to a travel angle $\lambda_f = 0$, while keeping the elevation constant. To do this, we formulate an open loop optimization problem over a finite horizon of N time steps.

The decision variables in this optimization problem are both the states and the control inputs over the horizon:

$$z = \begin{bmatrix} x_1^\top & x_2^\top & \dots & x_N^\top & u_0^\top & u_1^\top & \dots & u_{N-1}^\top \end{bmatrix}^\top,$$

where $x_k = [\lambda_k, r_k, p_k, \dot{p}_k]^\top$ is the state at step k and $u_k = p_{c,k}$ is the pitch reference input.

1.3.1 Objective Function

The goal is to minimize a cost function that balances tracking of the final travel angle and the effort in pitch control, given by:

$$\phi = \sum_{i=0}^{N-1} (\lambda_{i+1} - \lambda_f)^2 + q\, p_{c,i}^2,$$

where $q \geq 0$ is a weight that adjusts the trade-off between minimizing control effort and minimizing travel error. The results of varying this parameter are described in section 1.4. The cost function can be written in quadratic programming form:

$$\min_z \frac{1}{2} z^\top Q z + c^\top z,$$

where Q is a block-diagonal matrix containing the weights for each state and input over the horizon, and c is a vector of linear terms (zero in this case). When using the Matlab function `quadprog` we only needed to supply the matrix Q which was made in the following way:

```
Q1 = zeros(mx,mx);

Q1(1,1) = 2;
Q1(2,2) = 0;
Q1(3,3) = 0;
Q1(4,4) = 0;
P1 = 1;                    % Our q value.
Q = gen_q(Q1,P1,N,M);      % Generate Q (handout code)
c = zeros(mx*N+M*mu,1);
```

The optimization problem is subject to system dynamics and physical limitations:

1.3.2 Discrete-time dynamics

$$x_{k+1} = A_d x_k + B_d u_k, \quad k = 0, \ldots, N-1$$

These dynamics are enforced as linear equality constraints in the QP:

$$A_{eq} z = b_{eq},$$

where A_{eq} encodes the discrete dynamics over the horizon, and b_{eq} contains the initial condition x_0. The implementation looked like this:

```
% Generate A (handout code)
Aeq = gen_aeq(A1, B1, N, mx, mu);
beq = zeros(N*mx,1);
beq(1,1) = x1_0;
```

1.3.3 Pitch angle limits:

$$|p_k| \leq 60^\circ \frac{\pi}{180}, \quad k = 1, \ldots, N$$

This ensures the helicopter does not exceed physically safe pitch angles. Note that this applies to the actual pitch state p_k rather than the pitch reference p_c. This is because its the state we want to be within these bounds, and the reference and state is not nessesearly equal due to system dynamics.

```
% Bounds on input p_c
ul      = -60*pi/360;
uu      = 60*pi/360;

% Bounds on state. Zero except for pitch
xl      = -Inf*ones(mx,1);
xu      = Inf*ones(mx,1);
xl(3)   = ul;
xu(3)   = uu;
```

```
% Generate constraints (handout code)
[vlb,vub] = gen_constraints(N,M,xl,xu,ul,uu);

% We want the last input to be zero
vlb(N*mx+M*mu)  = 0;
vub(N*mx+M*mu)  = 0;
```

Solving this QP using MATLAB's `quadprog` yields the optimal trajectory of states x_k and control inputs u_k over the horizon.

1.4 The weights of the optimization problem

To compare how the trajectory changes with respect to q we tried three different values. We expected to get the following result:

- Small $q \rightarrow$ aggressive control to quickly reach λ_f, possibly using larger pitch angles.
- Large $q \rightarrow$ smoother control with smaller pitch commands, but slower convergence to the target.

We calculated the trajectories using `quadprog`. The travel for the different values of q are shown in Figure 1. Here we can clearly see that the smaller q values lead to a faster trajectory while the q value of 12 led to a longer time to reach the target.

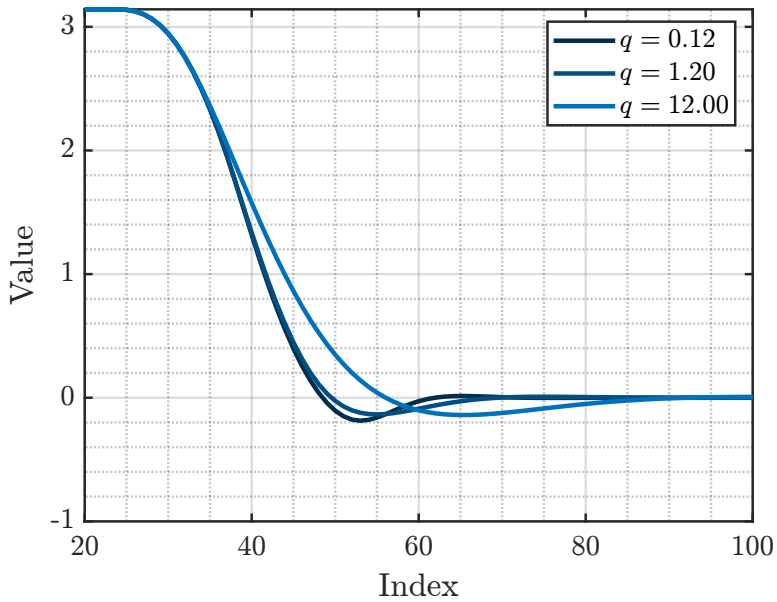

Figure 1: Travel for tree different q values

Similarly in Figure 2 when comparing the reference pitch we see that larger values of q led to less aggressive maneuvers.

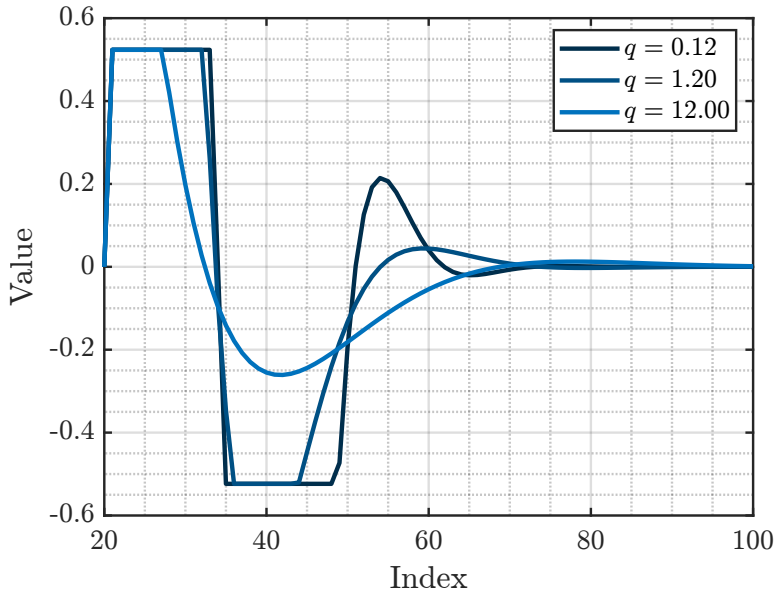

Figure 2: Pitch reference for different q values

1.5 Experimental results

The travel path from running the planned trajectory is shown in figure 3. As expected the travel starts out around π and is then pushed down towards 0. Firstly we observe that the travel does not reach 0 as planned and secondly it starts to rapidly diverge after the trajectory is finished. This is due to the open loop nature of our system and that we don't have any way to correct for modeling errors or noise.

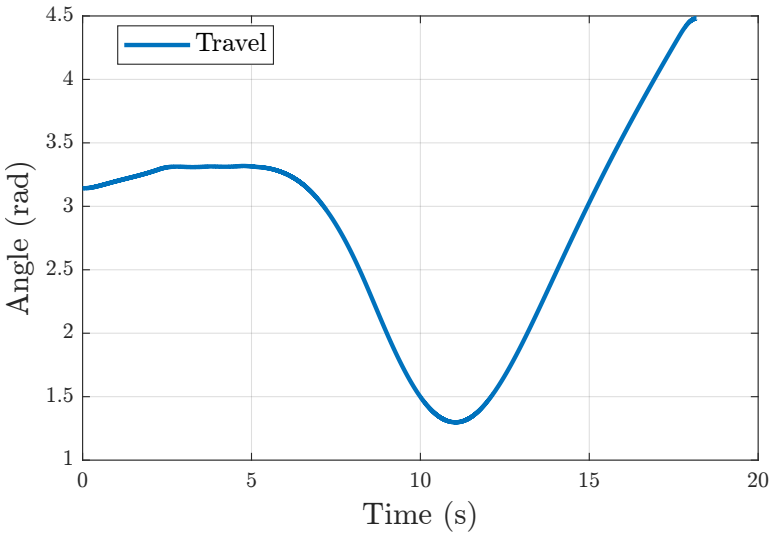

Figure 3: Experimental data from the open loop system

2 Optimal Control of Pitch/Travel with Feedback (LQ)

2.1 LQ controller

In this section we used an LQ controller to introduce feedback to mitigate the helicopter not reaching its reference in the last section. The introduction of feedback is done through the manipulated variable

$$u_k = u_k^* - K(x_k - x_k^*) \tag{2}$$

Where u^* and x^* are the optimal input sequences from the optimal trajectory that was calculated in the last part, and x_k is the measured state. K will in this case be the LQR gain, but we could have chosen any other method for finding the gain. This means that the input (which is the setpoint for the inner control layer) will change from u^* if the helicopter deviates from the calculated part, but if it follows x^* the input will be the same as what was calculated. The control hierarchy can be seen in figure 4

The LQ controller is a controller that minimizes the objective function

$$J = \sum_{i=0}^{\infty} \Delta x_{i+1}^\top Q \Delta x_{i+1} + \Delta u_i^\top R \Delta u_i, \quad Q \geq 0, R > 0 \tag{3}$$

Where $\Delta x = x - x^*$ and $\Delta u = u - u^*$ are the deviations in the states x and the inputs u from the optimal trajectory. Q and R are LQR weights, which are square matrices with dimensions corresponding to the amount of states and inputs, respectively.

The controller that minimizes this objective function is on the form

$$\Delta u = -K \Delta x \tag{4}$$

Which gives

$$u_k - u_k^* = -K(x_k - x_k^*) \tag{5}$$

Which if you move u_k^* to the other side you get equation 2

The K matrix was found by using `dlqr(A,B,Q,R)` in matlab, which works by solving the following equation

$$K = (R + B^\top P B)^{-1}(B^\top P A + N^\top) \tag{6}$$

where P is a positive definite solution to the discrete algebraic Riccati equation.

$$P = A^\top P A - (A^\top P B + N)(R + B^\top P B)^{-1}(B^\top P A + N^\top) + Q \tag{7}$$

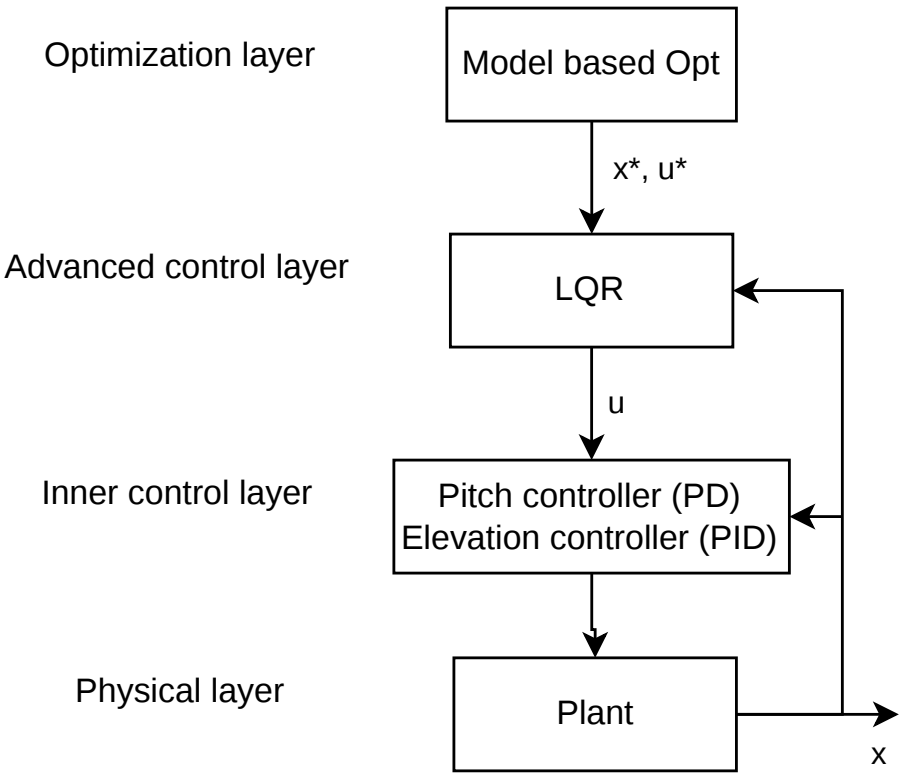

Figure 4: Illustration of control layers, strongly inspired by the lab-exercise text.

and A and B are system matrices from the discrete-time LTI system in section 1.2

The goal of the Q and R matrices is to weigh the different states and inputs against each other. If the R matrix is high for some state, it means that any deviation by the system from the setpoint will be punished harder in this state. If the Q matrix has a high value for an input, control action will be expensive, and the controller will try to not use that much control action in order to reach the setpoint.

It can be shown that it's only the ratio between the different values for the states in R and inputs in Q that matter, not the absolute values. So if you multiply both matrices by a constant, K will not change.

In our system we chose to use diagonal matrices for both Q and R. We started by just choosing

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{8}$$

and

$$K = 1 \tag{9}$$

2.2 Model Predictive Control

Instead of using an LQ regulator in order to implement feedback to the system one could use model predictive control (MPC) instead. This can be done in two ways, the most common way is to re-optimize from the new measured position for each time-step in order to get a new x^* and u^*. Then apply the new u^* to the system. This can be done by having a function that makes a new optimal prediction for every timestep.

Advantages with this is that you can get an optimal trajectory no matter where you end up in the future. The system with the LQR controller might end up taking an "older" optimal route if the helicopter for some reason ends up far away from the predicted path. The main disadvantage with this method is the increased computational cost when doing an online optimization.

Another way of implementing MPC would be by just switching out the LQ controller with an MPC. So you would stabilize it in the same way, but you would use an MPC to do it. Implementing this could be done by having an optimization that runs every timestep, but that uses the pre-calculated trajectory as it's setpoint. This would mean that the controller will prefer the initialy predicted trajectory. You might prefer using this method if you really want to follow the initially calculated trajectory and don't want it to drift from this, even though you'll end up at the same end position eventually.

You could possibly combine these, but that would end up being a cascade MPC controller.

In figure 5 you can see how these two methods would look like.

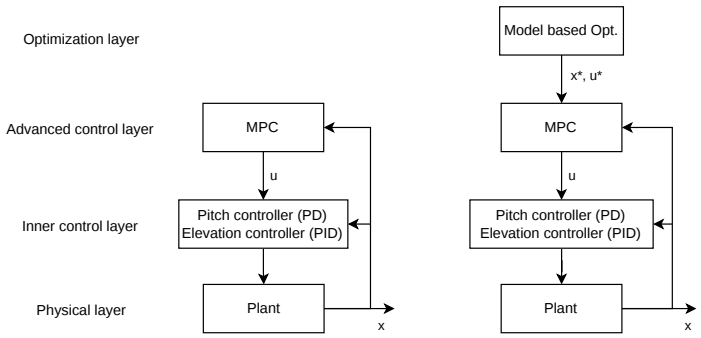

Figure 5: The left diagram shows how the first way of doing MPC would look. On the right side, the LQR from figure 4 is switched out with MPC.

2.3 Experimental results

When testing with feedback control we first ran the system with the matrices from 8 and 9. In Figure 6 we can see that the Helicopter with feedback follows the precalculated path a lot closer than without feedback, but that it still doesn't perfectly follow the path. Therefore we tried some other LQR parameters. We first changed to $R = 0.01$ and kept Q the same. This response was pretty similar, the result can be seen in Figure 7. We also tried increasing K and set it to $K = 100$. This response was a lot worse as shown in Figure 8. Therefore we sticked with $K = 1$. It would probably be possible to reach an even better response with some more tuning.

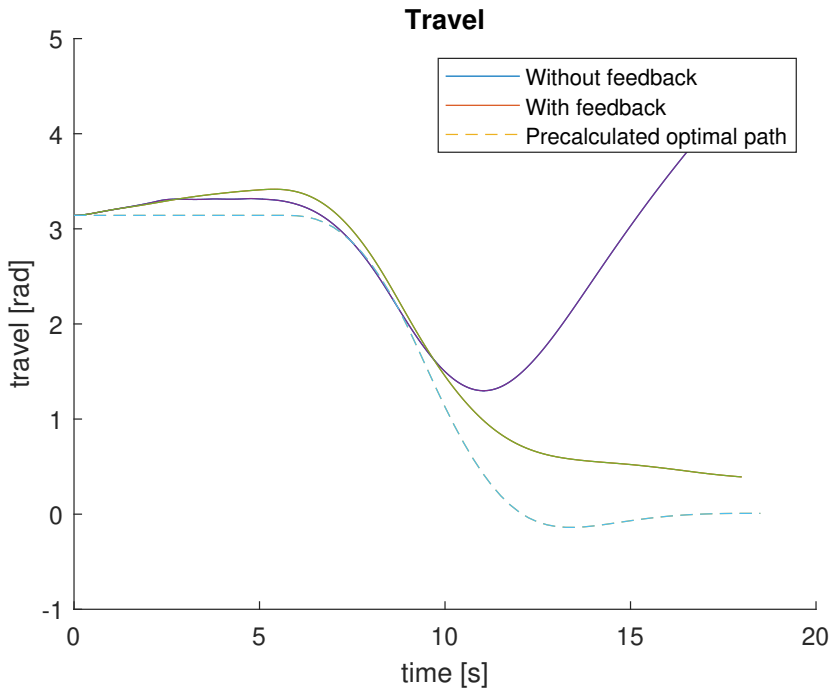

Figure 6: Travel of helicopter with feedback and tuning matrices from 8 and 9.

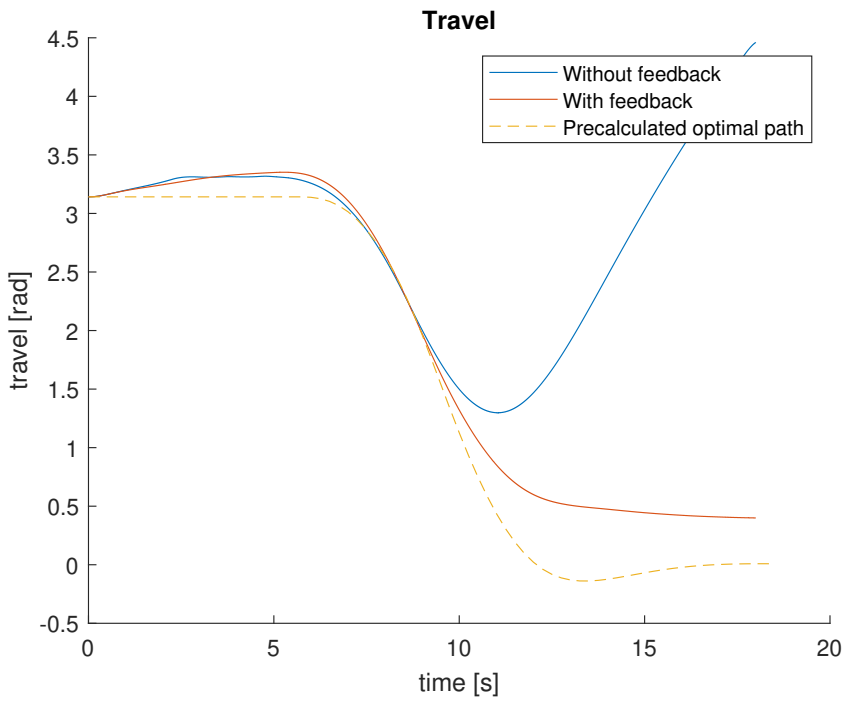

Figure 7: Helicopter travel with feedback. Matrix Q from 8 and $R = 0.01$

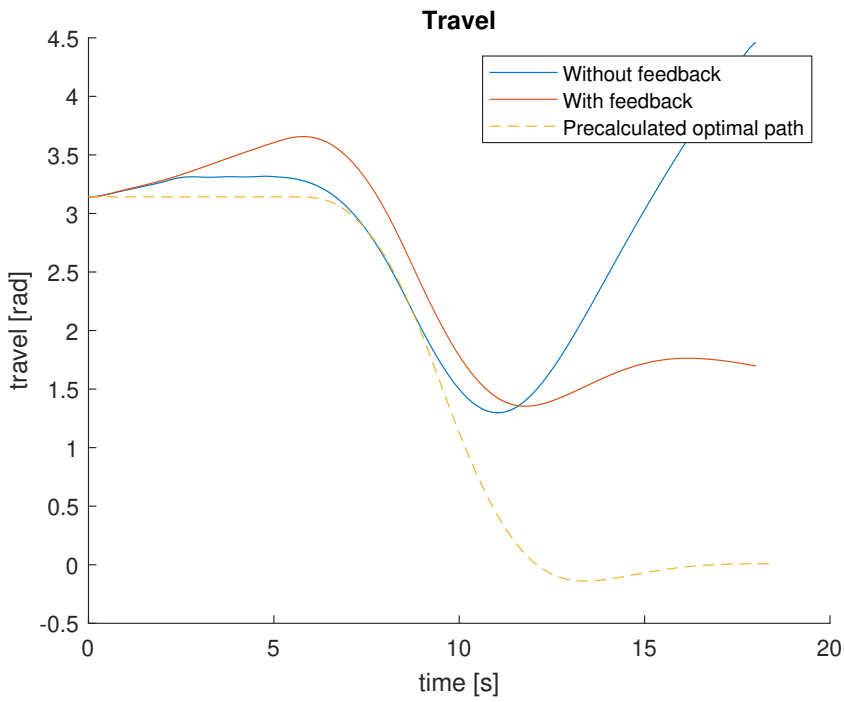

Figure 8: Helicopter travel with feedback. Matrix Q from 8 and $R = 100$

2.4 MATLAB and Simulink

The following script was used to find the lqr gain K.

```
q11 = 1;
q22 = 1;
q33 = 1;
q44 = 1;

R = 100;

Q = diag([q11 q22 q33 q44]);
K = dlqr(A1,B1,Q,R);
```

3 Optimal Control of Pitch/Travel and Elevation with Feedback

3.1 The continuous model

For the last lab-day we also wanted to control the elevation e and the elevation rate $\dot{e}$. We added the variables to the state vector x from equation (1a). The new state vector has the from

$$x = \begin{bmatrix} \lambda \\ r \\ p \\ \dot{p} \\ e \\ \dot{e} \end{bmatrix}, \quad u = \begin{bmatrix} p_c \\ e_c \end{bmatrix}$$

and the system matrices are now

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & -K_2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -K_1(K_{pp}+K_{pd}) & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & -K_3K_{ep} & -_{K3}K_{ed} \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ K_1K_{pp} & 0 \\ 0 & 0 \\ 0 & K_3K_{ep} \end{bmatrix}$$

3.2 The discretized model

The system was then again descreticed using the forward Euler method to get

$$x_{k+1} = A_d x_k + B_d u_k$$

with the system matrecies A_d and B_d given by

$$A_d = \begin{bmatrix} 1 & dt & 0 & 0 & 0 & 0 \\ 0 & 1 & -K_2dt & 0 & 0 & 0 \\ 0 & 0 & 1 & dt & 0 & 0 \\ 0 & 0 & -K_1K_{pp}dt & 1-K_1K_{pd}dt & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & dt \\ 0 & 0 & 0 & 0 & -K_3K_{ep}dt & 1-K_3K_{ed}dt \end{bmatrix}$$

$$B_d = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ K_1 K_{pp} dt & 0 \\ 0 & 0 \\ 0 & K_3 K_{ep} dt \end{bmatrix}$$

3.3 Optimization Problem

For the last lab-day our goal was to calculate an optimal trajectory over an obstacle. The obstacle was represented by the inequality contraint

$$e_k \geq \alpha \exp\left(-\beta(\lambda_k - \lambda_t)^2\right), \quad \forall k \in \{1, \ldots, N\}$$

Since we now control the elevation we also added the cost of elevation input use to the cost-function and thus the cost function is now given by

$$\phi = \sum_{i=0}^{N-1} \left[(\lambda_{i+1} - \lambda_f)^2 + q_1 p_{c,i}^2 + q_2 e_{c,i}^2\right]$$

3.4 Optimization problem in MATLAB

Implementing the non-linear constrains required us to add it for every timestep. This was done in the following way:

```
function [c, ceq] = non_linear_constraint(x, mx, N)
    alpha = 0.2;
    beta = 20;
    lambda_t = 2*pi/3;

    c = zeros(N,1);

    for k = 1:N
        c(k) = alpha * exp(-beta * (x(1 + (k-1)*mx) -
            lambda_t)^2) - x(5 + (k-1)*mx);
    end

    ceq = [];
end
```

The other constraints were added as in the previous sections and the new $q2$ value was added to the objective calculation. Lastly this was fed to MATLAB's `fmincon` function and the optimal path was calculated.

3.5 Experimental results

The experimental results show that the system was not able to follow the planned path. The travel angle was pretty nicely followed as seen in Figure 9, but the elevation shown in Figure 10 was a lot lower than what it was supposed to be. There may be many different reasons for this, but we believe that the main issue is that the system doesn't take into account that the pitch and elevation are coupled. This issue is discussed in part 3.6. The other reason why the system might be unable to follow the path is due to bad tuning of the LQ-controller. This might account for part of the error, but this much offset wouldn't have been only due to tuning.

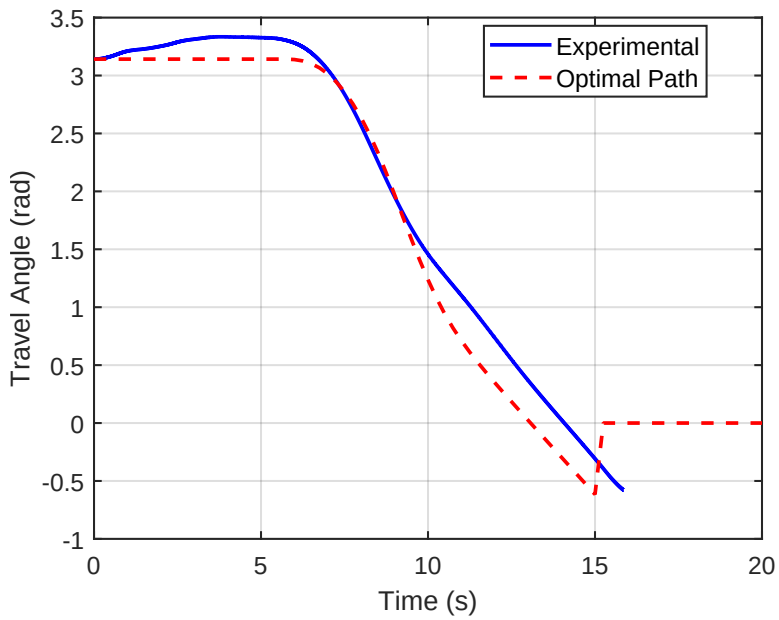

Figure 9: A comparison of the optimal pre-calculated travel and the experimental travel

Another thing that should be noted is that even the optimal path, calculated beforehand, violates the non-linear constraint. The reason for this is that the optimal path is not evaluated for enough timesteps. Since no timestep is lining up with the peak of the constraint function, the optimal path is passing right through the constraint. So increasing the amount of timesteps would solve this issue, but that would also take a lot more computing power. If this constraint was an actual mountain or some other obstacle that it was critical you didn't crash into, you could solve the issue by adding an extra "safety-area" (increasing the constraint) around the obstacle, as well as increasing the amount of timesteps.

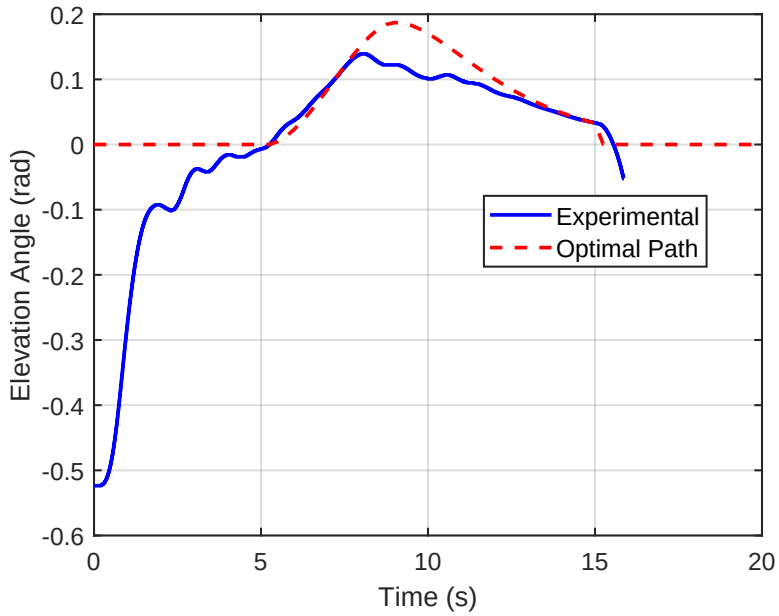

Figure 10: A comparison of the optimal pre-calculated elevation and the experimental elevation

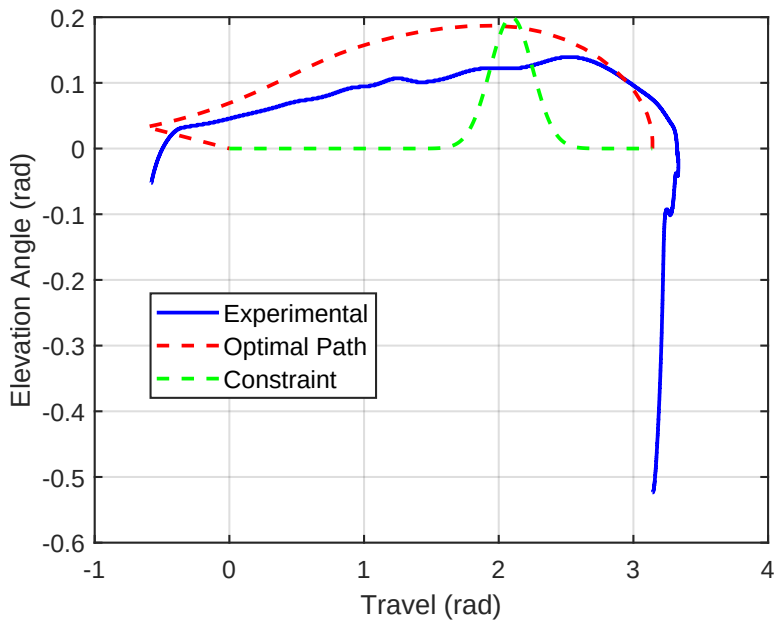

Figure 11: The actual path of the helicopter plotted against the constraint and the pre-calculated optimal path. (The experimental path starts at -0.5 radians because it starts on the floor.)

3.6 Decoupled model

As noted in the last section, the control of the elevation is always going to be somewhat off, due to the relationship between pitch and elevation not being included in our model. The true relationship is

$$\ddot{e} = C_p V_s \cos(p) + C_e \cos(e) \tag{10}$$

.

But why isn't the helicopter crashing when this is not accounted for? There are two main reasons for this:

1. The controllers from the advanced and inner layer shown in figure 5 implement feedback so any deviation from the optimal trajectory will cause the controllers to try and correct it.

2. There is a pitch contraint set in the SQP optimizer makes sure the helicopter never wants to pitch too much and thus we never loose all thrust.

But we still have an issue, especially if the helicopter, as discussed in the last section, has an obstacle it strictly can't crash into.

For solving this we will discuss three different options: One of them is to rewrite equation 1a to

$$\ddot{e} = C_p V_s \cos(p) + C_e \cos(e),$$

and write the dynamics in state-space form:

$$\dot{x} = f(x, u).$$

This would make the whole system dynamics non-linear and would force us to redesign the whole control system using methods from non-linear control as the LQ-controller doesn't work for non-linear dynamics. Although this would use more computing power, this is the best solution.

Another solution would be adding this sum

$$\sum_{k=1}^{P} \left(\dot{e}_k - \dot{e}_{k-1} - \Delta t C_p V_s \cos(p_k) - \Delta t C_e \cos(e_k)\right)^2$$

to the objective function.

At first glance, this seems like it could be a good solution. It works by punishing deviation from the real dynamics when calculating the optimal path, so that the actual path will be physically possible. But since this only applies to the planned path, it's not going to make the helicopter follow the

path any better when actually flying, this would also add complexity for our solver, but as we're already using a non-linear solver it can probably handle it. All in all this is a bad solution.

The last solution we will discuss would be adjusting the Q-weight for $\dot{e}$ in the LQ controller based on the pitch angle $q = q_e \cos(p)$. With this solution we are basically ignoring the $\dot{e}$ refrence more and more as we enter the non-linear areas. This would make it easier for the LQ controller to correct the deviations in e because the $\dot{e}^*$ optimal trajectory will not limit it as much. This seems like a solution that could improve the performance, but it would still not solve the underlying issue. This solution could also make the system unstable, as if the helicopter is falling quickly, i.e. if p approaches $\frac{\pi}{2}$ the controller stops caring about $\dot{e}$ which can be dangerous if it is falling down. Another drawback is that you have to recompute the LQ matrices at every timestep. This makes this solution somewhat unviable.

References

[1] Norwegian University of Science and Technology, Department of Engineering Cybernetics. Ttk4135 optimization and control helicopter lab, 2025.